CODEBASE GUIDE

How a request flows through the orchestration platform

A plain-English walkthrough, written to be read beside the code

Project Dynamic AI Agent Orchestration Platform

Covers 25 backend modules, the mock MCP server, the React frontend, the test suite

Assumes No prior knowledge of MCP, LangGraph, DAGs, embeddings or server-sent events

> **How to read this.** Part 4 is the heart of the document: one real request traced from the browser to the answer and back, step by step. If you only read one thing, read that. Parts 1–3 prepare you for it; Part 5 is a reference you dip into when a particular file confuses you.

Contents

Part 1 · What this system actually does

You type a sentence. The system works out who should answer it, invents the specialists needed, gives each one exactly the tools it is allowed to touch, runs them in the right order, and writes you one answer. You can watch the whole thing happen live.

1.1 The problem it solves

Imagine you ask a question about your team's work:

> `"Predict the velocity for our next sprint based on previous Jira sprints."`

Answering that properly needs several different skills. Someone has to fetch the sprint history from Jira. Someone has to do the arithmetic on it. Someone might need to check who is on holiday next month. Someone should sanity-check the result before you trust it.

The obvious way to build this is one big AI assistant with a very long instruction sheet and every tool bolted on. That approach breaks down in three ways:

- **The instruction sheet grows forever.** Every new skill means editing one enormous block of text, and instructions start to contradict each other.
- **Tool choice gets worse as tools are added.** Given forty tools, a model picks the wrong one more often than it does with four.
- **You cannot explain what happened.** When the answer is wrong, there is no point in the process you can inspect and say "here is where it went astray".

This project does the opposite. It splits the work across several small specialists, each with a narrow job and a short list of tools, and it records every decision so you can inspect it afterwards.

1.2 What makes it unusual

Most systems like this keep a list of pre-written specialists in a configuration file: “here is the Jira agent, here is its instruction text”. This project does **not**. There is no file anywhere in the repository containing an agent's instructions.

Instead, the configuration file describes **what a class of agent is allowed to do** — which tools it may touch, which AI models it may use, what rules it must obey — and the system **writes the agent itself, freshly, for each request**.

Here is the same idea in two pictures. The configuration says only this:

```yaml
# backend/config/agents.yaml
- id: jira_reporting
  description: Retrieves and summarizes historical Jira sprint records…
  allowed_tool_selectors: ["jira.get_sprints", "jira.get_sprint_issues"]
  allowed_models: ["claude-sonnet-5"]
  max_iterations_limit: 8
  requires_approval: false
  policy: |
    Never state a figure you did not retrieve.
```

Notice what is missing: there is no `system_prompt`. That is deliberate, and the system refuses to start if you add one. When a real request arrives, the planner writes an agent to fit it. This is genuine output from a real run of this system:

```
# written at runtime, exists in no file
name          : velocity_forecaster
role          : Projects next-sprint velocity from completed points
system_prompt : "Work from actual Jira data, never from assumed numbers.

                 Method:
                 1. Call jira.get_sprints to retrieve the most recently
                    closed sprints. Use up to the last 6 closed sprints…
                 3. Show the arithmetic explicitly: list the completed-point
                    values you are averaging, the sum, the count, and the
                    resulting mean…"
tool_selectors: [jira.get_sprints, jira.get_sprint_issues]
```

The planner invented the name, wrote the method, and asked for two tools. The configuration allowed those two tools, so it got them. Had it asked for the tool that *writes* to Jira, it would have been refused.

1.3 The one idea to hold on to

> **Configuration owns permissions. The planner owns instructions.** Neither can do the other's job. A written agent can ask for *fewer* tools than its configuration allows, never more. That single rule is what makes it safe to let an AI write another AI's instructions.

Everything else in this codebase — the graph, the event stream, the database, the approval gate — is machinery in service of that idea. If you keep it in mind, the rest of the code stops looking arbitrary.

Part 2 · Vocabulary decoder

Every term this project uses, explained with an everyday comparison first. Skim this now, come back when a word trips you up.

2.1 Agent, tool, capability

AGENT

An AI model plus a set of instructions plus a list of tools it may use, pointed at one job. Think of a temporary contractor: you tell them what to do, hand them the specific keys they need, and they work until the job is done.

In this project: an agent is created for a single task and then discarded. It is not a class in the code — it is a record built at runtime by `backend/app/agents/composer.py`.

TOOL

A function the AI can call, with a name, a description, and a list of arguments. The AI does not run the function itself; it says *"please call get_sprints with limit=6"*, our code runs it, and hands back the result.

In this project: tools come from an MCP server (see below). One example is `jira.get_sprints`, which returns recent sprint records.

CAPABILITY — ALSO CALLED AN "ENVELOPE"

A permission slip. It describes a *class* of agent: which tools that class may touch, which models it may use, how many steps it may take, and any rule it must always follow. It contains no instructions, because instructions are written per request.

In this project: the four capabilities live in `backend/config/agents.yaml`: `jira_reporting`, `capacity_analysis`, `analysis_only` and `jira_writeback`. The word "envelope" is used because an agent must fit *inside* it.

2.2 MCP — the wall socket for tools

MCP (MODEL CONTEXT PROTOCOL)

Think of a wall socket. Any appliance with the right plug works, and the socket does not care what the appliance is. MCP is that socket, but for AI tools. A separate program (an "MCP server") says *"here are the tools I offer, here are the arguments each one takes"*, and any AI application can plug in and use them without knowing anything about it in advance.

In this project: at startup we connect to every server listed in `backend/config/mcp_servers.yaml` and ask what tools it has. We believe the answer. No tool name or argument list is written in our Python anywhere. That is why swapping the fake Jira server for a real one is a configuration edit, not a code change.

The mock server in `mcp_servers/jira_mock/server.py` offers four tools: three that read sprint data and one that writes a plan back to the board. It reads from a JSON fixture file, so the whole system can be demonstrated without real Jira credentials.

2.3 DAG, fan-out, superstep

DAG (DIRECTED ACYCLIC GRAPH)

A to-do list where some items must wait for others. "Directed" means the arrows point one way: A must finish before B. "Acyclic" means no loops — A cannot wait for B while B waits for A, because nothing would ever start.

In this project: the planner produces a DAG of tasks. Fetching the sprint history has no dependencies, so it starts immediately. Forecasting depends on the fetch, so it waits.

FAN-OUT

Starting several jobs at the same time instead of one after another. If two tasks both only depend on the fetch, they can run side by side once the fetch is done. That is a fan-out.

In this project: `route_ready_tasks()` in `backend/app/graph/build.py` works out which tasks are ready and starts all of them at once.

SUPERSTEP

One round of work. The system does everything it can do right now, waits for all of it to finish, then looks again at what has become possible. Each round is a superstep.

In this project: after every superstep the dispatcher recalculates which tasks are now unblocked. A four-task plan might run in three supersteps: fetch, then two tasks together, then the final review.

2.4 LangGraph — graphs, nodes, state, checkpoints

LANGGRAPH

A library for describing a process as a set of boxes connected by arrows, then running it. You define the boxes ("nodes") and the rules for moving between them; LangGraph handles running them, possibly several at once, and remembering where it got to.

In this project: the graph has exactly four nodes no matter how many agents run. See `backend/app/graph/build.py`.

NODE

One box in the graph — just a function. It receives the current state, does its job, and returns the bits of state it wants to change.

In this project: the four nodes are `planner`, `dispatcher`, `worker` and `synthesizer`.

STATE

A shared notepad that every node can read and write. When two nodes run at the same time and both write to it, a "reducer" function decides how to combine their writes so nothing is lost.

In this project: the notepad is `OrchestrationState` in `backend/app/graph/state.py`. Two reducers exist: one merges each worker's output into a dictionary, one appends cost records to a list.

CHECKPOINTER

A save point. After every step, the current state is written to a database. If the process stops — because it is waiting for a human, or because it crashed — it can be picked up from exactly where it left off.

In this project: checkpoints go to PostgreSQL. This is what makes the approval pause survive a server restart.

`SEND` AND `INTERRUPT()`

`Send` means "run this node with this specific input", and returning several of them starts several copies at once. `interrupt()` means "stop here, save everything, and wait until someone tells me to continue".

In this project: `Send` creates the parallel workers; `interrupt()` is the human approval gate in `backend/app/graph/worker.py`.

2.5 Embeddings and pgvector

EMBEDDING

A way of turning text into a list of numbers, so that texts about similar things end up with similar numbers. Once text is numbers, "which of these is most like that?" becomes arithmetic instead of guesswork.

In this project: `backend/app/embeddings.py` asks OpenAI's `text-embedding-3-small` to turn each capability description into 384 numbers. When your goal arrives it is turned into 384 numbers too, and we compare, to decide which capabilities to offer the planner — and to find similar past runs.

Be aware: if `OPENAI_API_KEY` is not set, the platform falls back to a local vectorizer that compares shared *words* rather than meaning. It still runs, but a goal worded differently from a past run will not find it — measured over reworded queries, top-1 retrieval is 5/6 with the real model against 3/6 on the fallback. The fallback exists so the test suite and offline development never need a key.

PGVECTOR

An add-on for PostgreSQL that lets the database store those lists of numbers and answer "find me the five most similar rows" efficiently.

In this project: each finished run stores its goal's embedding. When a new goal arrives we look for similar past goals and show them to the planner, so it can reuse an approach that worked.

2.6 SSE — how the browser watches

SSE (SERVER-SENT EVENTS)

A one-way live feed from server to browser. The browser opens a connection and leaves it open; the server pushes messages down it whenever something happens. Like a news ticker: the server talks, the browser listens.

In this project: the browser opens `GET /runs/{id}/events` and receives each step as it happens. The feed **replays everything already sent** before going live, so you can attach late — or even after the run finished — and still see the whole story.

Exactly seven event names exist, and nothing else is ever sent:

```
run.started    task.started    tool.called    tool.result
approval.required    run.completed    run.failed
```

2.7 async and await, in one page

> **`ASYNC` / `AWAIT`**
>
> A way for one program to juggle many slow jobs. Think of a waiter with several tables: after taking an order they do not stand and watch the kitchen, they go and serve another table. `await` marks the moment where waiting would happen — "this will take a while, go do something else and come back".
>
> **In this project:** almost everything is `async`, because almost everything waits on something slow: an AI model, a database, an MCP tool. It is what lets two agents genuinely run at the same time.

You will see this shape constantly. It means "start this, and let other work continue while it runs":

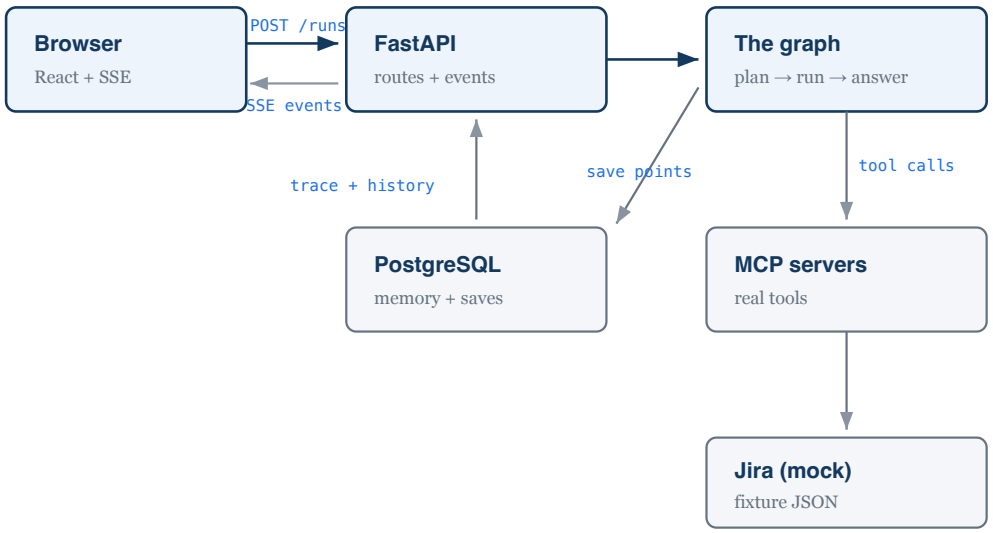
```
async def run_task(task_id, objective):
    response = await bound_model.ainvoke(messages)   # waits for the AI
```

Part 3 · The big picture

3.1 The five moving parts

Before tracing a request, get the cast in your head. There are five, and everything in the codebase belongs to one of them.

Part	What it is	Where it lives
The browser app	A React page where you type a goal and watch the run unfold.	`frontend/src/`
The API	A FastAPI web server. Accepts goals, streams progress, serves results.	`backend/app/api/`, `backend/app/main.py`
The brain	The planner that designs agents, and the graph that runs them.	`backend/app/graph/`, `backend/app/agents/`
The hands	MCP servers offering real tools. Here, a fake Jira reading a JSON file.	`backend/app/mcp/`, `mcp_servers/jira_mock/`
The memory	PostgreSQL. Stores save points, run history, and goal embeddings.	`backend/app/memory/`



The five parts and what passes between them.

3.2 What happens at startup

Before any request can be served, `backend/app/main.py` builds four things once, in this order. If any of them fails, the server refuses to start — deliberately, because a half-configured orchestrator is worse than one that will not boot.

1 **Read the capability registry**

`load_capability_registry()` · `backend/app/agents/registry.py`

Reads `agents.yaml`, checks every record is valid, and turns each capability description into numbers for later searching. A typo in that file stops the server here, with a message naming the file and the offending record.

2 **Connect to the MCP servers and ask what they offer**

`McpManager.discover_tools()` · `backend/app/mcp/manager.py`

For each server in `mcp_servers.yaml`, start it, connect, and call `list_tools()`. Whatever comes back *is* the tool list. A server that fails to start is reported and skipped rather than killing the whole platform.

3 **Prepare the database**

`create_schema()` · `backend/app/memory/db.py`

Turn on the pgvector extension and create the tables if they do not exist. Safe to run on every boot.

4 **Build and compile the graph**

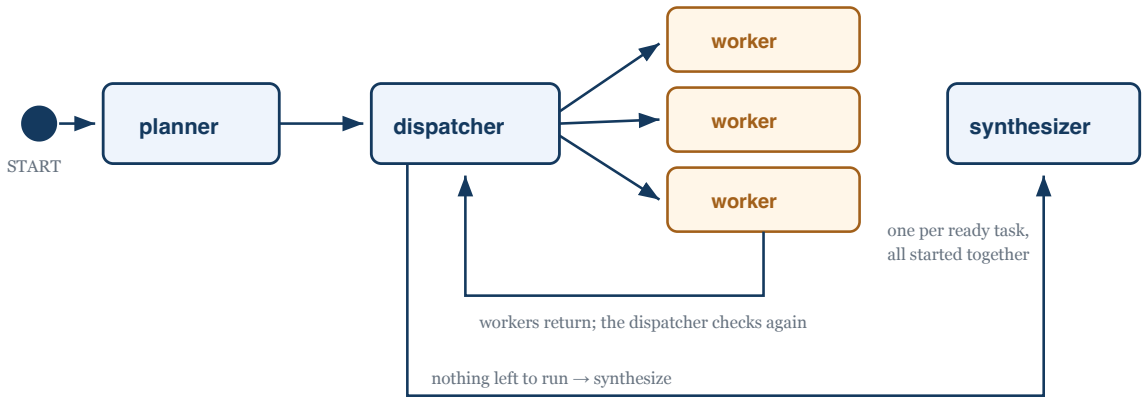
`build_orchestration_graph()` · `backend/app/graph/build.py`

Wire the four nodes together and attach the PostgreSQL checkpointer. The result is stored on the app so every request uses the same compiled graph.

> **Worth noticing:** everything built at startup is *infrastructure* — the permission registry, the tool list, the graph skeleton. No agent exists yet. Agents are created per request, and that distinction is the whole design.

3.3 The shape of the whole system

The graph has four nodes and never changes shape. What changes is the *plan*, which decides how many workers run and in what order.

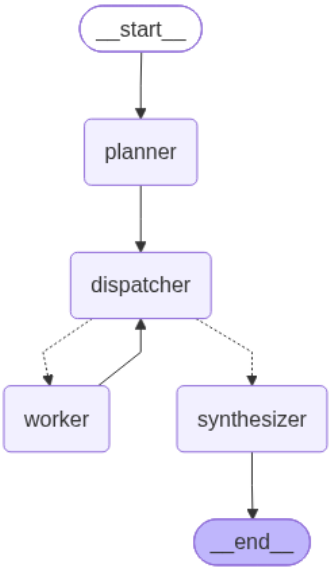


Four fixed nodes. The dispatcher is the scheduler: after every round it asks "what can run now?" and starts all of it at once, or moves on to the synthesizer.

This is the point worth saying out loud: **parallelism is a property of the plan, not of the graph.** A plan with four independent tasks runs four workers side by side without a single line of the graph changing.

The same graph, generated from the code

The diagram above is drawn by hand, so it can drift away from the code — the project's main architecture diagram once did exactly that. This one cannot: it is emitted from the compiled object itself by `docs/render-graph.py`, so it can only ever show the topology the application actually built.



Generated from `compiled.get_graph()`. The dotted edges out of `dispatcher` are the conditional ones — that is the scheduler. The `worker → dispatcher` arrow is a real cycle.

Regenerate it with `.venv/bin/python docs/render-graph.py`, which also writes `docs/graph.mmd` — the Mermaid source, which GitHub renders natively in the README and the architecture document.

Part 4 · One request, end to end

This is the heart of the guide. Everything below is a real request that really ran, with the real numbers it produced. Open the files as you go.

4.1 The request we are following

```
Goal:  "Predict the velocity for our next sprint based on previous Jira sprints."
Run:   run_20de22af70364a22
Cost:  $0.1238   (15,704 tokens in, 4,819 out)
```

The planner decided one agent was enough for this goal, created it, and it called two tools. We will follow all seventeen steps. Section 4.3 then covers the more complex path where the system pauses to ask a human for permission.

4.2 Steps 1–17, browser to answer

1 You type a goal and press submit

`frontend/src/components/GoalInput.jsx`

A plain form. It holds the text you typed and calls back to `App.jsx` when you submit. It knows nothing about the network.

2 The app sends the goal to the server

`createRun()` · `frontend/src/api/client.js`

Every network call in the whole frontend lives in this one file — components never call `fetch` themselves. It sends `POST /api/runs` with your goal. The Vite dev server forwards `/api` to the backend, so the browser only ever uses same-origin URLs and there is no API address to configure.

3 The server accepts the goal and returns immediately

`create_run()` · `backend/app/api/routes_runs.py`

This is important: the server does **not** wait for the run to finish. It creates a run ID, starts the work in the background, and replies straight away with `202 Accepted` and that ID. A full run can take a minute; no HTTP request should be held open that long.

```
run_id = await request.app.state.orchestrator.start_run(payload.goal)
return CreateRunResponse(run_id=run_id)
```

4 A row is written and the work is scheduled

`Orchestrator.start_run()` · `backend/app/orchestrator.py`

Two things happen. A `runs` row is inserted with your goal and its embedding, so future runs can find this one. Then the graph is scheduled as a background task.

The reference to that task is deliberately kept in a set: Python can garbage-collect a background task nobody is holding, and it would vanish mid-run.

5 The browser opens the live feed

`useRunStream()` · `frontend/src/hooks/useRunStream.js`

With the run ID in hand, the browser opens `GET /runs/{id}/events` and holds it open. Because the feed replays what it has already sent, it does not matter that steps 3 and 4 may already have happened.

6 The planner looks for similar past runs

`find_similar_runs()` · `backend/app/memory/retrieval.py`

Your goal is turned into numbers and compared against every completed run in the database. The closest few are shown to the planner as "here is how similar questions were handled". If the database is unreachable this returns nothing rather than failing — memory is a help, not a requirement.

7 A menu of capabilities is retrieved

`search_by_capability()` · `backend/app/agents/registry.py`

The four capabilities are ranked by how close their descriptions are to your goal. For a velocity question, `jira_reporting` ranks top.

The menu shows each capability's allowed tools **resolved to real names**, not as patterns. This matters: the planner's request is checked against resolved names, so showing it patterns would invite it to ask for something it cannot have.

8 The planner designs the agents

`plan_goal()` · `backend/app/graph/planner.py`

One call to the AI, with the goal, the capability menu, the tool descriptions and the recalled runs. It must reply in a fixed shape: a list of tasks, each with a capability, an objective, its dependencies, and **the agent to create for it**.

For our run it returned a single task, and this agent:

```
capability_id : jira_reporting
agent.name    : velocity_forecaster
agent.role    : Projects next-sprint velocity from completed points
agent.tools   : [jira.get_sprints, jira.get_sprint_issues]
agent.prompt  : "Work from actual Jira data, never from assumed numbers.
                 Method:
                 1. Call jira.get_sprints to retrieve the most recently
                    closed sprints…
                 3. Show the arithmetic explicitly…
                 4. Check for a trend… Only call it a trend if the
                    movement is sustained across sprints…"
```

Nothing in that block exists in any file. It was written for this request.

9 The plan is checked twice, and retried once if wrong

`validate_plan() and check_envelopes()` · `planner.py, state.py`

First the structure: are the capability IDs real, are dependencies resolvable, are there duplicate task IDs, is there a loop? Then each designed agent is trial-fitted to its capability: are the tools it asked for allowed, is the model allowed, is the step count within the limit?

If either check fails, the error is fed back to the AI and it gets **one** retry. A second failure fails the run loudly.

> Checking here, at planning time, is a deliberate choice. An agent that reaches too far costs one retry, instead of a run that dies halfway through having already paid for earlier tasks.

10 `run.started` reaches your browser

`build_event_recorder()` · `backend/app/orchestrator.py`

The first event goes out, carrying the goal, the whole plan, the planner's reasoning, and which capabilities were considered. Every event does two things at once: it is pushed to the live feed *and* written to the `steps` table. A trace that exists in one but not the other would be worse than neither.

11 The dispatcher works out what can run

`route_ready_tasks()` · `backend/app/graph/build.py`

It compares the plan against what has finished and returns one `Send` per ready task — starting them all at once — or the string `"synthesizer"` when nothing is left. Our plan has one task, so one worker starts.

12 The agent is actually created

`realize_agent()` · `backend/app/agents/composer.py`

This is where the designed agent becomes a real one. The envelope is enforced again — the worker does not trust the plan it was handed — and the capability's policy is appended to the written prompt:

```
Work from actual Jira data, never from assumed numbers.
Method: 1. Call jira.get_sprints…

Non-negotiable constraints (these override anything above…):
Use the provided tools to fetch real numbers. Never estimate,
invent or recall a figure you did not retrieve in this task.
```

The policy goes **last**, on purpose. Later instructions carry more weight, so a rule placed first could be talked over by whatever the planner wrote.

13 The worker runs its tool loop

`build_worker()` · `backend/app/agents/factory.py`

A plain loop, written by hand rather than borrowed from a framework, so that three things stay visible: the step limit is enforced, every tool call is traced, and every token is counted. It repeats up to `max_iterations` times:

- Ask the AI what to do next.
- If it asked for no tools, its text is the answer — return it.
- If it asked for tools, call them, add the results to the conversation, loop.

Running out of iterations is an explicit failure, not a shrug: returning the last text would be a guess.

14 A tool is called for real

`call_tool_with_policy()` · `backend/app/mcp/manager.py`

Our agent called `jira.get_sprints`, then `jira.get_sprint_issues`. Each call is wrapped in a timeout and up to three attempts with a growing pause between them.

Retries are for connection failures and timeouts only. A tool that *answers* with an error has done its job — retrying spends the same money for the same answer — so the error goes back to the agent, which decides what to do about it.

Two events go out per call: `tool.called` then `tool.result`.

15 The worker's answer goes into shared state

`build_worker_node()` · `backend/app/graph/worker.py`

The result is written to the shared notepad under this task's ID, along with a record of what it cost. Because several workers may write at once, the reducers in `state.py` combine them safely.

Control returns to the dispatcher, which looks again. Nothing is left, so it routes to the synthesizer.

16 The synthesizer writes the final answer

`synthesize_answer()` · `backend/app/graph/synthesizer.py`

One AI call over the goal and every task's output. It has no tools on purpose: it must answer from what the agents actually retrieved, and say so when something was not retrieved.

It also writes a two-sentence summary. That summary is stored and embedded, so a future similar goal can be shown "this is how that one went".

17 The run is closed and the browser renders

`_complete_run()` · `orchestrator.py` → `runState.js`

The `runs` row is updated with the answer, the summary and the total cost, and `run.completed` goes out carrying the answer and every task's output.

In the browser, `reduceEvent()` folds that event into the display state, and the answer is rendered as Markdown.

> **A detail that confuses people.** There is no `task.completed` event — the seven names are fixed. So the browser *works out* when a task finished: a task is done when a task depending on it starts, or when the run completes and reports every task's status. That logic is `ancestorsOf()` in `frontend/src/hooks/runState.js`, and its docstring explains why.

4.3 The second path: pausing for a human

Some work should not happen without permission. The `jira_writeback` capability writes to a shared board, so it is marked `requires_approval: true`. Here is a real run of *"Work out our next sprint commitment and publish it to the Jira board"*:

```
velocity     jira_reporting      velocity_forecaster       [get_sprints, get_sprint_issues]
capacity     capacity_analysis   capacity_reader           [get_team_capacity]
commitment   analysis_only       commitment_risk_reviewer  []
publish      jira_writeback      sprint_plan_publisher     [publish_sprint_plan]
```

Four agents, all created at runtime, across four capabilities. The first two have no dependency on each other, so they ran side by side. Then:

A **The worker stops before it does anything**

`interrupt()` · `backend/app/graph/worker.py`

Because the capability demands approval, the worker calls `interrupt()` *before* running the agent. LangGraph saves the entire state to PostgreSQL and stops. Nothing is held in memory, so the pause survives a server restart.

`requires_approval` is read from the capability, never from the written agent — so a generated agent cannot approve itself.

B **You are shown what you are approving**

`_pause_for_approval()` · `orchestrator.py` → `ApprovalDialog.jsx`

An `approval.required` event carries the agent name, its capability, its tools and **the generated prompt that is about to run** — because that prompt is what you are actually approving, and it exists in no file.

The run's spend so far is recorded too, so a run sitting on an approval queue does not report that it cost nothing.

C **You decide, and the run continues**

`approve_run()` · `routes_runs.py` → `resume_run()`

`POST /runs/{id}/approve` resumes the graph from its save point. Approve, and the write happens. Decline, and the task is recorded as declined, the tool is never called, and the run still finishes — reporting honestly what was skipped:

> "It was **not published to the Jira board** — the publish step
> was declined by a human, so nothing was written."

Part 5 · Component reference

Every module, in the same four-part shape, so you can jump straight to whichever file is confusing you. Read the **What to look at first** line if you are in a hurry.

> Every module in `backend/app/` opens with a docstring that says what it does *and what it deliberately does not do*. Those docstrings are the most reliable documentation in the repository. This section expands on them.

5.1 Startup and configuration

main.py

`backend/app/main.py` · 119 lines

WHAT IT DOES

Builds every long-lived object once at startup, in dependency order, and hands them to the routes. Also closes them cleanly on shutdown.

WHY IT EXISTS

Something has to own the process lifecycle. Keeping it in one file means there is exactly one place to look to see what the server is made of.

KEY FUNCTIONS

`lifespan()` — the whole startup and shutdown sequence. `read_health()` — the liveness check Docker uses.

WHAT TO LOOK AT FIRST

The order of statements inside `lifespan()`. It reads like a recipe: registry, tools, database, checkpointer, graph.

settings.py

`backend/app/settings.py` · 57 lines

WHAT IT DOES

Reads every environment variable into one frozen object: API key, database URL, model names, timeouts, how many capabilities to show the planner.

WHY IT EXISTS

So no other module reads `os.environ`. If you want to know what is configurable, this file is the complete answer, and `.env.example` mirrors it.

KEY FUNCTIONS

`get_settings()` — cached, so the whole process shares one instance.

WHAT TO LOOK AT FIRST

The comment above `planner_model`: platform-level models are configuration, while worker models come from the capability. That split is intentional.

5.2 The capability registry

models.py

`backend/app/agents/models.py` · 173 lines

WHAT IT DOES

Defines the three shapes an "agent" takes as it comes into being: **CapabilitySpec** (what is configured), **SynthesizedAgent** (what the AI wrote), and **AgentSpec** (the finished, approved agent).

WHY IT EXISTS

These three types are the spine of the whole design. Configuration owns permissions (`CapabilitySpec`), the planner owns instructions (`SynthesizedAgent`), and only the composer can turn the second into the third.

KEY FUNCTIONS

Validators. A capability ID must be snake_case; a description must be more than five words, because it is searched and a bare job title retrieves badly; the default model must be in the allowed list.

WHAT TO LOOK AT FIRST

The docstring on `SynthesizedAgent`: "every field here is a *request*". That is the mental model — it is untrusted input, not a decision.

registry.py

`backend/app/agents/registry.py` · 116 lines

WHAT IT DOES

Loads `agents.yaml`, validates it, embeds each capability description, and answers "which capabilities best match this goal?"

WHY IT EXISTS

It is the list of capability IDs that legally exist. The planner's output is checked against this object, so a broken registry must fail at startup, not at request time.

KEY FUNCTIONS

`load_capability_registry()` — read and validate, with error messages that name the file and the offending record. `search_by_capability()` — rank by similarity. `get()` — look up one, or raise listing what was available.

WHAT TO LOOK AT FIRST

The last line of `search_by_capability()`. If nothing scores above zero it still returns a menu, because a planner with no options cannot plan.

5.3 The MCP layer

manager.py

`backend/app/mcp/manager.py` · 202 lines

WHAT IT DOES

Reads `mcp_servers.yaml`, starts and holds a connection to each server for the life of the process, discovers their tools, and enforces timeout and retry policy on every call.

WHY IT EXISTS

This is the boundary between our code and the outside world. It is also why no tool definition is written in Python: whatever the servers advertise is what we have.

KEY FUNCTIONS

`load_server_connections()` — config to connection details. `discover_tools()` — ask every server what it offers. `call_tool_with_policy()` — one call, with timeout and retries.

WHAT TO LOOK AT FIRST

`_hold_session()`, and the long comment above the class explaining it. See the warning below — it is the least obvious code in the repository.

> **Why each connection gets its own background task.** The MCP library is built on a system where a connection must be closed by the same task that opened it. But a web server runs startup and shutdown in *different* tasks. Opening a connection at startup and closing it at shutdown therefore crashes with *"attempted to exit cancel scope in a different task"*.
>
> The fix: give each connection its own small task that opens it, waits for a stop signal, and closes it. Both ends then happen in one task. This was found by hitting the error, not by reading documentation.

registry.py (tools)

`backend/app/mcp/registry.py` · 120 lines

WHAT IT DOES

Keeps an index of every discovered tool under a namespaced name like `jira.get_sprints`, resolves patterns to real tools, and ranks tools against a goal.

WHY IT EXISTS

Namespacing keeps two servers from colliding if both offer a `search` tool. And `select()` is the function that turns a permission pattern into an actual list — the mechanism the whole safety boundary rests on.

KEY FUNCTIONS

`select()` — patterns to tools. `search()` — rank by similarity to the goal. `get()` — one tool by name.

WHAT TO LOOK AT FIRST

`select()`, and its comment: an empty pattern list means *no tools*, which is deliberately different from meaning *all tools*.

5.4 Creating an agent

planner.py

`backend/app/graph/planner.py` · `192 lines`

WHAT IT DOES

Turns your goal into a plan of purpose-built agents: retrieves the menu, makes one AI call that designs an agent per task, validates the result, retries once on failure.

WHY IT EXISTS

A classifier that maps a goal to one agent cannot express "fetch, then forecast and check capacity in parallel, then review". That is a graph, not a label. So the routing decision produces a plan.

KEY FUNCTIONS

`plan_goal()` — the whole pipeline including the retry. `render_capability_menu()` — the menu, with tools resolved to real names. `check_envelopes()` — trial-fit every designed agent.

WHAT TO LOOK AT FIRST

`PLANNER_SYSTEM_PROMPT`. Its eight numbered rules are the contract the planner works under, in plain English.

composer.py

`backend/app/agents/composer.py` · `119 lines`

WHAT IT DOES

Decides whether a designed agent is allowed to exist, and if so builds the finished `AgentSpec` with the capability's policy appended to its prompt.

WHY IT EXISTS

This is the security boundary. It is the reason it is safe to let an AI write another AI's instructions. Part 6 covers it in detail.

KEY FUNCTIONS

`realize_agent()` — every check, in order. `compose_system_prompt()` — written prompt first, policy last.

WHAT TO LOOK AT FIRST

The three lines computing `escalation`. That subtraction of two sets is the entire permission system.

factory.py

`backend/app/agents/factory.py` · 181 lines

WHAT IT DOES

Takes a finished `AgentSpec` plus its tools and returns a runnable function that executes the tool-calling loop.

WHY IT EXISTS

This is where "agents are not classes" becomes literal. `build_worker()` closes over a spec and returns a coroutine. There is no agent class to instantiate.

KEY FUNCTIONS

`build_worker()` — returns the runnable. `_invoke_tool()` — one tool call, with its two trace events.

WHAT TO LOOK AT FIRST

The docstring's three reasons for hand-writing the loop instead of using a prebuilt agent: the step limit is enforced, every call is traced, every token is attributed. A prebuilt agent would hide all three.

5.5 The graph

state.py

`backend/app/graph/state.py` · 158 lines

WHAT IT DOES

Defines the shared notepad, the shape of a plan, the reducers that make concurrent writes safe, and the rule for what may run next.

WHY IT EXISTS

Everything that flows between nodes is defined here, in one place, so the contract cannot drift.

KEY FUNCTIONS

`ready_tasks()` — the single definition of "what can run next". `validate_plan()` — structural checks including cycle detection. `_merge_outputs()` and `_append_records()` — the reducers.

WHAT TO LOOK AT FIRST

`ready_tasks()` — four lines, and the entire scheduling rule.

build.py

`backend/app/graph/build.py` · 149 lines

WHAT IT DOES

Wires the four nodes together and compiles the graph.

WHY IT EXISTS

The graph's shape is fixed and tiny; the plan decides how much runs and in what order. This file is where that trick is set up.

KEY FUNCTIONS

`route_ready_tasks()` — the scheduler. `build_planner_node()` and `build_orchestration_graph()` — wiring.

WHAT TO LOOK AT FIRST

`route_ready_tasks()`. Returning a list of `Send` starts many workers at once; returning `"synthesizer"` ends the loop.

worker.py

`backend/app/graph/worker.py` · 179 lines

WHAT IT DOES

Runs exactly one task: creates the agent from its envelope, pauses for approval if required, runs it, and records the result.

WHY IT EXISTS

It is the bridge between the plan and the factory, and the place the approval gate lives.

KEY FUNCTIONS

`build_worker_node()` — the node itself. `_objective_with_context()` — quotes upstream output into this task's prompt. `_declined_result()` and `_failed_result()`.

WHAT TO LOOK AT FIRST

`_objective_with_context()`. This is how data travels along a dependency arrow: a finished task's answer is quoted into the next task's prompt, instead of both fetching it.

synthesizer.py

`backend/app/graph/synthesizer.py` · 119 lines

WHAT IT DOES

Turns every task's output into one answer for you, plus a short summary stored as memory.

WHY IT EXISTS

Four agent outputs are not an answer. Something must combine them and say plainly what was and was not established.

KEY FUNCTIONS

`synthesize_answer()` — two AI calls, answer then summary. `render_task_outputs()` — lays outputs out in plan order.

WHAT TO LOOK AT FIRST

`SYNTHESIZER_SYSTEM_PROMPT`, rule 2: use only figures that appear in the agent outputs. It has no tools, so it cannot fill a gap by fetching — it must report the gap.

5.6 Orchestration and events

orchestrator.py

`backend/app/orchestrator.py` · 147 lines

WHAT IT DOES

Runs the graph for one request in the background, publishes events, and records the outcome.

WHY IT EXISTS

It joins three things that know nothing about each other: the graph, the live event feed, and the database.

KEY FUNCTIONS

`start_run()` — create the row, schedule the work, return the ID. `resume_run()` — continue after approval. `build_event_recorder()` — publish and store each event. `_pause_for_approval()`, `_complete_run()`, `_fail_run()`.

WHAT TO LOOK AT FIRST

The docstring's explanation of why the approval event is published *here* rather than inside the worker: a node emitting its own event would emit it a second time when the graph replays that node on resume.

events.py

`backend/app/events.py` · 117 lines

WHAT IT DOES

An in-memory message board. Holds the seven event names, fans each event out to every open browser connection, and keeps a replay buffer.

WHY IT EXISTS

The replay buffer is the important part. Without it, a fast run could finish before the browser opened its connection, and the page would hang on a run that was already over.

KEY FUNCTIONS

`publish()` — record and fan out. `subscribe()` — replay history, then go live. `history()`.

WHAT TO LOOK AT FIRST

The module docstring's honesty about the trade: this does not survive a restart or spread across multiple servers. The durable copy is the `steps` table; this is only the live view.

5.7 Memory and the database

Memory comes in three tiers, which is worth seeing as a whole before reading the files:

Tier	Stored where	Used for
Working	LangGraph's PostgreSQL checkpointer	Resuming a paused or crashed run from its save point
Episodic	`runs` and `steps` tables	Replaying a trace, cost history, what the UI reads
Semantic	pgvector column on `runs`	Finding similar past runs to show the planner

db.py

`backend/app/memory/db.py` · 56 lines

WHAT IT DOES

Creates the database connection, and creates the tables and the pgvector extension on boot.

WHY IT EXISTS

One place owns the connection. It also converts the connection string into the slightly different format LangGraph's checkpointer needs, so both are configured from one variable.

KEY FUNCTIONS

`create_schema()`, `build_engine()`, `to_checkpointer_dsn()`.

WHAT TO LOOK AT FIRST

The docstring's admission that tables are created directly rather than with migrations, and why that is acceptable here but not in production.

models.py (database)

`backend/app/memory/models.py` · 87 lines

WHAT IT DOES

Defines two tables. `runs` is one row per goal: what was asked, planned, answered and spent. `steps` is the durable copy of every trace event.

WHY IT EXISTS

`steps` is what makes a finished run replayable after the in-memory feed has forgotten it.

KEY FUNCTIONS

`Run.as_summary()` — the compact view fed to a later planner.

WHAT TO LOOK AT FIRST

The `goal_embedding` column — this is the semantic tier, and the only unusual column type in the schema.

retrieval.py

`backend/app/memory/retrieval.py` · 172 lines

WHAT IT DOES

Every database read and write outside the checkpointer: starting a run, recording steps, pausing, finishing, and finding similar past runs.

WHY IT EXISTS

So database access lives in one file and nothing else writes SQL.

KEY FUNCTIONS

`start_run()`, `record_step()`, `record_plan()`, `pause_run()`, `finish_run()`, `get_run()`, `find_similar_runs()`.

WHAT TO LOOK AT FIRST

Why `pause_run()` is separate from `finish_run()`: a paused run is not over, and stamping it with a completion time would make an interrupted run look finished.

5.8 The HTTP API

Method	Path	What it does
POST	`/runs`	Submit a goal; returns a run ID at once
GET	`/runs/{id}`	Final state, plan, answer, cost, full trace
GET	`/runs/{id}/events`	The live feed
POST	`/runs/{id}/approve`	Resume a paused run
GET	`/agents`	The capability registry, for the UI
GET	`/tools`	Tools discovered at startup
GET	`/docs`	Interactive API documentation, generated

routes_runs.py

`backend/app/api/routes_runs.py` · 87 lines

WHAT IT DOES

The four run endpoints. Deliberately thin: validate input, call the orchestrator or the memory layer, return.

WHY IT EXISTS

To turn orchestration into HTTP, and nothing more. No logic lives here.

KEY FUNCTIONS

`create_run()`, `read_run()`, `stream_run_events()`, `approve_run()`.

WHAT TO LOOK AT FIRST

`stream_run_events()`, and its docstring on replay. Also note `approve_run()` refuses with a conflict error if the run is not actually paused.

routes_registry.py & schemas.py

`backend/app/api/` · 45 + 97 lines

WHAT THEY DO

`routes_registry.py` serves `/agents` and `/tools` — read-only views of what was loaded. `schemas.py` defines every request and response shape.

WHY THEY EXIST

These endpoints are the evidence that routing is configuration-driven: the UI can show you exactly what the planner is choosing from. The schemas become the generated documentation at `/docs`, so their descriptions *are* the API documentation.

KEY FUNCTIONS

`list_agents()`, `list_tools()`.

WHAT TO LOOK AT FIRST

The comment on `AgentResponse` explaining why it keeps its old field names even though it now describes capabilities: the UI is built on them, and they still read correctly.

5.9 Supporting modules

embeddings.py

`backend/app/embeddings.py` · 91 lines

WHAT IT DOES

Turns text into 384 numbers using OpenAI's `text-embedding-3-small`, falling back to a local lexical vectorizer when no key is configured.

WHY IT EXISTS

Both the capability search and the past-run memory need to compare texts. Keeping it local means tests are deterministic and the system works offline.

KEY FUNCTIONS

`embed_text()`, `embed_texts()` (one batched call), `cosine_similarity()`, `uses_hosted_embeddings()`.

WHAT TO LOOK AT FIRST

The comment on re-normalizing. The model is natively 1536 dimensions; asking for 384 keeps the stored vectors the same width as the database column, but a truncated vector is no longer unit length — and `cosine_similarity()` is a bare dot product, so without re-normalizing it would rank by magnitude as well as direction.

costs.py

`backend/app/costs.py` · 79 lines

WHAT IT DOES

Turns token counts into dollars, using a table of prices per million tokens, and adds them up.

WHY IT EXISTS

So a run can report what it cost. Every AI call is priced and tagged with the task that spent it.

KEY FUNCTIONS

`price_response()`, `sum_usage()`, and the `Usage` record which can be added together.

WHAT TO LOOK AT FIRST

`unpriced_models`. A model missing from the price table is recorded by name rather than silently costed at zero — a wrong number is worse than a visible gap.

chat_models.py

`backend/app/chat_models.py` · 39 lines

WHAT IT DOES

Builds an AI client from a model name. One function.

WHY IT EXISTS

So the planner, the workers and the synthesizer all build clients the same way, and a test can substitute a fake at a single point.

KEY FUNCTIONS

`build_chat_model()`.

WHAT TO LOOK AT FIRST

The missing-key error. It fails here with an actionable message, rather than letting an empty key surface as an authentication error three layers deep.

5.10 The mock Jira MCP server

server.py

`mcp_servers/jira_mock/server.py` · 178 lines

WHAT IT DOES

A standalone MCP server offering four tools over a real MCP connection, backed by a JSON fixture: three that read sprint data, one that writes a plan back.

WHY IT EXISTS

So the platform can be demonstrated without Jira credentials — and so the approval gate guards something real. A human-approval step on a read-only system would be theatre.

KEY FUNCTIONS

`get_sprints()`, `get_sprint_issues()`, `get_team_capacity()`, `publish_sprint_plan()`.

WHAT TO LOOK AT FIRST

Any tool's docstring. It becomes the tool's description that the AI reads, so the `Returns:` section is not decoration — it is how the agent knows what it is getting back.

schemas.py & generate_fixtures.py

`mcp_servers/jira_mock/` · 114 + 197 lines

WHAT THEY DO

`schemas.py` types every tool's return value, which makes the server publish a proper output schema. `generate_fixtures.py` regenerates the fixture data.

WHY THEY EXIST

The types catch a malformed fixture on the way out. The generator guarantees the data is internally consistent: an issue list always adds up to its sprint's recorded totals.

KEY FUNCTIONS

`build_issues()` — enforces consistency by construction. `check_consistency()` — verifies it anyway before writing.

WHAT TO LOOK AT FIRST

The `schemas.py` docstring, which explains that the output schema reaches other MCP clients and our own code, but **never the AI** — so the prose docstrings are still the only channel to the model.

5.11 The frontend

client.js

`frontend/src/api/client.js` · 85 lines

WHAT IT DOES

Every network call the app makes. Components never call `fetch` themselves.

WHY IT EXISTS

One file to look at to see the app's entire relationship with the server.

KEY FUNCTIONS

`createRun()`, `getRun()`, `approveRun()`, `streamRun()`, `listAgents()`, `listTools()`.

WHAT TO LOOK AT FIRST

`streamRun()`. Because the server names every event, the browser needs a separate listener per name — the default handler never fires.

runState.js

`frontend/src/hooks/runState.js` · 133 lines

WHAT IT DOES

Folds the stream of events into the state the page renders. Pure JavaScript, no React.

WHY IT EXISTS

It is the only non-trivial logic in the frontend, so it is kept separate where it can be reasoned about on its own.

KEY FUNCTIONS

`reduceEvent()` — one event in, new state out. `ancestorsOf()` — every task a given task depends on, directly or indirectly.

WHAT TO LOOK AT FIRST

The docstring on per-task completion. Because there is no `task.completed` event, a task is marked done when something depending on it starts. That is derived, not received.

The components

`frontend/src/components/` · 6 files

WHAT THEY DO

GoalInput takes your goal. **TraceTimeline** shows each task and its tool calls as they happen. **RunGraph** draws the plan as a diagram. **CostPanel** shows tokens and dollars. **ApprovalDialog** asks you to approve or decline. **Markdown** renders model output as formatted text.

WHY THEY EXIST

To make the run visible. The trace is the product as much as the answer is.

KEY FUNCTIONS

One component per file, named for the file, each a plain function.

WHAT TO LOOK AT FIRST

`Markdown.jsx`, and its note that raw HTML is deliberately **not** enabled: this text comes from an AI by way of tool output, so anything injected stays inert.

Part 6 · The safety boundary

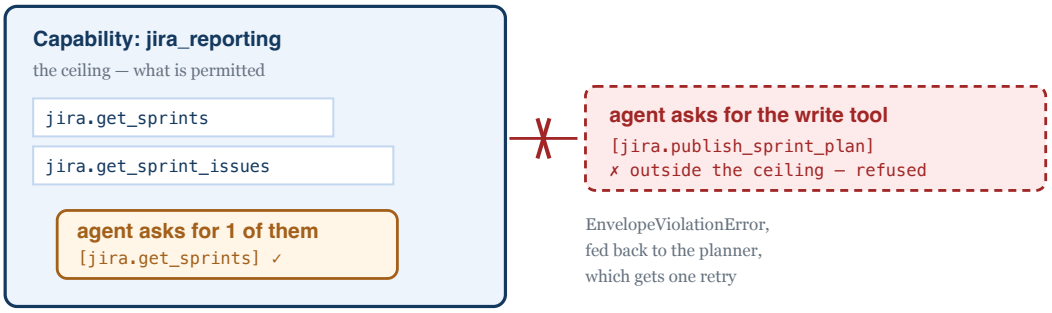
One file, about thirty lines, is what makes the whole design defensible. It is worth understanding properly.

6.1 Why a written agent is dangerous

We let an AI write another AI's instructions and choose its tools. Stated plainly, that sounds reckless. Consider what could go wrong:

- The planner asks for the tool that **writes** to Jira on a task that should only read.
- It writes an agent that tells itself to ignore the safety rules.
- It asks for an expensive model, or a hundred steps.
- It quietly clears the flag that requires human approval.

Every one of these is blocked, and none of them relies on the AI behaving well. The protection is structural: the capability is a **ceiling**, and a written agent can only ask for a subset of it.



An agent may narrow its envelope. It can never widen it.

6.2 The subset check, line by line

This is the core of `realize_agent()` in `backend/app/agents/composer.py`:

```
permitted = _resolved_names(tools, capability.allowed_tool_selectors)
requested = _check_selectors_resolve(capability, synthesized.tool_selectors, tools)

escalation = requested - permitted
if escalation:
    raise EnvelopeViolationError(…)
```

Read it slowly, because a subtle detail decides whether it works:

> **The comparison is on resolved tool names, never on the patterns.** A written agent could ask for the pattern `jira.*`. As *text*, that is not a subset of the capability's `jira.get_*` — a string comparison would either wrongly allow it or wrongly reject it. But asking the tool registry what each pattern actually resolves to, and comparing those two sets of real names, is exact. A string-prefix check here would be the bug that lets a generated agent award itself the write tool.

Three further things come from the capability and are never read from the written agent:

Taken from the capability	Why it must not come from the agent
`requires_approval`	Otherwise a generated agent could approve itself and write to Jira unsupervised.
`allowed_models`	Stops an agent picking an expensive model, or one not cleared for the task.
`max_iterations_limit`	Caps how long an agent can loop, and therefore what it can spend.

Finally, the capability's `policy` is appended to the written prompt, under a heading that says it overrides everything above it. The order is deliberate: later instructions carry more weight with a language model, so a policy placed *first* could be talked over by whatever the planner wrote. Placed last, it has the final word.

6.3 Defence in depth

The same check runs in two places, for different reasons:

Where	When	Why there
The planner check_envelopes()	Before anything runs	So a violation costs one retry, instead of a run that dies halfway through having already paid for earlier tasks.
The worker build_worker_node()	Just before the agent runs	Because the worker should not trust a plan payload it was handed. If the check is reached here at all, something altered the plan after planning.

Both are tested. `backend/tests/test_composer.py` contains twelve attempts to escape an envelope, and every one must be refused.

Part 7 · Reading the tests as documentation

The tests are the most precise description of how this system is supposed to behave. When a docstring and a test disagree, believe the test.

7.1 What each test file proves

File	Lines	What it pins down
`test_registry.py`	122	A broken `agents.yaml` fails with a message naming the file and the record. Also that putting a `system_prompt` in configuration is **rejected** — the architecture is enforced, not just documented.
`test_composer.py`	147	The safety boundary. Twelve escalation attempts, all refused: the write tool, a widening pattern, a disallowed model, too many steps, clearing its own approval flag.
`test_planner.py`	251	The planner cannot invent a capability, cannot escape an envelope, cannot emit a loop or a duplicate task, and gets exactly one retry.
`test_factory.py`	145	An agent is bound only the tools its patterns match, the step limit is enforced, and a failing tool is reported to the agent rather than crashing the run.
`test_end_to_end.py`	439	A whole run against the real mock server: the event sequence, parallel dispatch, that the agents which ran exist in no config file, and that declining approval blocks the write.

Two tests are worth opening first, because they state the design in the clearest terms:

> `test_a_glob_cannot_widen_the_envelope` — the one that justifies comparing resolved names instead of patterns.
>
> `test_the_agents_that_ran_exist_nowhere_in_config` — the central claim of the whole project, asserted directly.

7.2 How the LLM is faked

Tests never call a real AI. They would be slow, cost money, and give different answers each time. `backend/tests/stubs.py` provides a stand-in whose replies are decided by a function you supply.

```
def responder(messages):
    system = # ... the system prompt this call was made with
    if "final answer for a user" in system:
        return AIMessage(content="**Forecast: 37 points.**")
    if "Fetch the closed sprints" in system:
        return AIMessage(tool_calls=[{"name": "get_sprints", …}])
```

One object serves the planner, every worker and the synthesizer, by branching on which system prompt it was called with. Notice what that implies: the test can match on prompt text *because the prompts are now part of*

the plan rather than the configuration.

Everything else in the tests is real — the actual MCP server starts as a subprocess, real tools are called, real fixture data comes back. The AI is the only thing faked.

Part 8 · Where do I look when…

8.1 Task-to-file index

I want to…	Go to
Add a new kind of agent	`backend/config/agents.yaml` — add a capability. No Python.
Add a new integration or tool source	`backend/config/mcp_servers.yaml`
Change how the planner behaves	`PLANNER_SYSTEM_PROMPT` in `graph/planner.py`
Understand why an agent was refused a tool	`realize_agent()` in `agents/composer.py`
See why two tasks did or did not run in parallel	`ready_tasks()` in `graph/state.py`
Change what a task can spend	`max_iterations_limit` in `agents.yaml`
Add or change an event	`EventName` in `app/events.py` — but see the warning below
Change what the browser shows	`frontend/src/hooks/runState.js`, then the component
Change model prices	`MODEL_PRICES_PER_MTOK` in `app/costs.py`
Make a tool require approval	Move it to a capability with `requires_approval: true`
Change tool timeouts or retries	`MCP_TOOL_TIMEOUT_SECONDS` / `MCP_TOOL_MAX_ATTEMPTS` in `.env`
Change the fake Jira data	`mcp_servers/jira_mock/generate_fixtures.py`, then re-run it
See what a past run actually did	`GET /runs/{id}`, or the `steps` table
Find the prompt an agent ran under	That run's `task.started` event — it exists nowhere else

> **Three interfaces are load-bearing.** The capability registry, the planner's output shape, and the seven event names. Changing any of them ripples through the backend, the tests and the browser at once. They are called out in `CLAUDE.md` for the same reason.

Part 9 · Gotchas, FAQ and weak points

9.1 Things that confuse people

"Where are the agents defined?"

They are not. `agents.yaml` defines *capabilities* — permission slips. The agents are written by the planner for each request and exist only in that run's trace. The filename is a leftover from an earlier design; the top-level key inside it is `capabilities`.

"The graph only has four nodes. Where does the parallelism come from?"

From the plan. The `dispatcher` node returns one `Send` per ready task, which starts that many copies of the `worker` node at once. A four-task plan runs four workers without the graph changing.

"Why is there no `task.completed` event?"

The seven event names are a fixed contract. The browser derives completion instead: a task is done once something depending on it starts, or when the run finishes and reports every task's status. The cost is a little cleverness in `runState.js`; the benefit is that the event vocabulary stays small and stable.

"Why does the same goal give different prompts each time?"

Because the prompts are written fresh per request. That is the design, and it is also its main cost — see the weak points below.

"Why is `/agents` returning capabilities and not agents?"

Because capabilities are the only thing that exists before a request arrives. The endpoint keeps its name and field names because the UI is built on them and they still read correctly.

"What is `_hold_session` doing?"

Working around a real constraint: an MCP connection must be closed by the task that opened it, but a web server starts and stops in different tasks. See the warning in §5.3.

9.2 Honest weak points

Worth knowing before you review, because they are deliberate trades rather than oversights:

Weakness	The trade
Prompts vary between runs	The direct cost of writing agents at runtime. A bad generation is possible. Bounded by the capability `policy`, and every run records the exact prompt it used, so any run can be audited afterwards.
Planning costs more	The planner writes prompts now, so it produces far more output than when it only picked from a list.
Retrieval degrades without a key	Without `OPENAI_API_KEY` the local fallback compares words, not meaning: 5/6 versus 3/6 top-1 retrieval on reworded queries. Changing embedding model also invalidates stored vectors, so past runs rank arbitrarily until re-run.
The live feed is in-process	It does not survive a restart or spread across servers. The durable trace in `steps` does.
No authentication	Anyone who can reach the API can approve a paused run.
Planner quality is untested	The tests pin the *contract* — it cannot invent an ID or escape an envelope — not the judgement. Since the planner now also writes prompts, this gap is larger than it was.
Tables are created directly	No migration tool. Fine for a single-service project with no deployed history.

9.3 Running it yourself

The fastest way to understand the flow is to watch one happen.

```
# everything, in containers
cp .env.example .env        # put a real ANTHROPIC_API_KEY in it
docker compose up --build

# then open
http://localhost:5173       # the app
http://localhost:8000/docs  # the API documentation
```

```
# the tests — no API key needed, the AI is faked
cd backend && pytest -q
```

Four goals worth trying, in order:

#	Goal	What to watch for
1	*Predict the velocity for our next sprint based on previous Jira sprints.*	The core path. Look at the created agent's name and prompt.
2	*How many story points did we complete in the last three sprints?*	A small goal gets a small plan — one agent, not four.
3	*Two independent things: a velocity forecast, and the team capacity available. Then review the combined picture for risk.*	Parallel fan-out. Two tasks start before either finishes.
4	*Work out our next sprint commitment and publish it to the Jira board.*	The run pauses. You are shown the generated prompt before you approve it.

> **The single most instructive thing you can do:** run the same goal twice and compare `plan[].agent.system_prompt` in `GET /runs/{id}` between the two runs. The agents are written fresh each time. That is the whole project in one comparison.

Companion documents: `README.md` for setup and demo scenarios · `docs/architecture.md` for design rationale · `CLAUDE.md` for the rules the code is held to.
Regenerate this PDF with `.venv/bin/python docs/render_guide_pdf.py`.